L-sweeps for the sparsified Lippmann–Schwinger equation

Notes on the `Lsweeps` branch of `SparsifyAndSweep.jl`, September 2026

1 Setting, and what limits the layered sweep

The package solves the two-dimensional Lippmann–Schwinger (LS) equation for the Helmholtz scattering problem,

$$u(x)+\omega^2\int_\Omega G(x-y)\,m(y)\,u(y)\,dy \;=\; -\omega^2\int_\Omega G(x-y)\,m(y)\,u_I(y)\,dy, \qquad G(x)=\tfrac{i}{4}H_0^{(1)}(\omega|x|), \tag{1}$$

with $m=1-1/c^2$ compactly supported in $\Omega=(0,1)^2$, discretised by a Nyström rule on a uniform grid with $h=\lambda/8$ and a corrected diagonal weight. The dense system $(I+\omega^2KM)u=g$ is applied by a circulant embedding and solved by GMRES; the preconditioner is that of Liu and Ying [1], in two stages.

Sparsify. A single 9-point stencil α is fitted so that α^* annihilates the columns of K centred outside the 3×3 neighbourhood μ, $\min_{\|\alpha\|=1}\|\alpha^*K_{\mu,\mu^c}\|$, and $\beta^*=\alpha^*K_{\mu\mu}$. Applying α^* to each row of the dense system gives a sparse surrogate $H\tilde{u}=f$, with rows $\alpha^*\tilde{u}_{\mu_i}+\omega^2\beta^*[m\tilde{u}]_{\mu_i}$ on the grid and rows $\gamma_i^*\tilde{u}_{\mu_i}$ on b layers of perfectly matched layer (PML) around it. The γ stencils are fitted to annihilate complex-stretched plane waves $e^{i\kappa\, r\cdot x^\sigma}$ at the local frequency $\kappa=\omega\sqrt{1-m(x)}$, so they are valid *inside* the medium, not only on the outer boundary. Inverting H exactly costs 3 GMRES iterations at every frequency tried; everything above 3 comes from the second stage.

Sweep. H is block tridiagonal over slices of width b along x_1. The moving-PML sweep approximates each Schur complement by a local problem on the slice padded with γ rows, and applies the resulting block LDU factorisation from both ends towards the middle. This is the bottleneck for parallelism: the application is a linear recurrence over the $\ell=n/b$ slices. Two fronts give concurrency two, and no amount of hardware shortens the recurrence. Measured on this code, the intrinsic speed-up of one application over slices is 1.8–1.95 in 2D and 1.6 in 3D, and for a single right-hand side the whole solve cannot gain more than about 4× from slice parallelism, because setup is half the time and the application is capped at two.

2 L-sweeps in a nutshell

Taus, Zepeda-Núñez, Hewett and Demanet [2] remove the recurrence by changing the decomposition. The domain is cut into a Cartesian array of $q\times r$ boxes Ω_{ij}, each extended by a thin layer and a PML into a local problem with matrix A_{ij}. Information moves between boxes through *polarized wavefields*: given the values of a field on the two grid layers $\Gamma_1\,|\,\Gamma_2$ on either side of an interface, one local solve reproduces the field beyond the interface and vanishes before it. Sources anywhere are handled in three stages: a local solve in every box; four *cardinal* sweeps, right, left, up and down, along the rows and columns of boxes; and four *diagonal* sweeps, corner to corner, in which the boxes on one anti-diagonal are updated together from an L-shaped trace. Each sweep is a

chain, but the chains are independent: the q rows in a horizontal sweep, the r columns in a vertical one, the $\min(q, r)$ boxes of an anti-diagonal in a diagonal one. With one rank per row of boxes and boxes of fixed size in wavelengths, the application costs $O(N/p)$ on $p = O(n)$ ranks, with only nearest-neighbour messages of one interface layer each. For a constant medium and a thick enough PML the preconditioner is a direct solver (one iteration); in general the iteration count grows like $\log \omega$.

The price is work: nine local solves per box per application against two slab solves per slice for the layered sweep, and up to four shifted decompositions with a partition of unity when a source sits on the skeleton of the decomposition. What is bought is a critical path of $O(q)$ box solves of $O(1)$ size instead of $O(\ell)$ slab solves of $O(n)$ size.

3 The transfer to the sparsified system

3.1 The identity is algebraic, and H satisfies its hypothesis

Order the unknowns of a local matrix A as $(\Omega_1, \Gamma_1, \Gamma_2, \Omega_2)$, where Γ_1 and Γ_2 are the layers adjacent to the interface on each side. If the stencil couples only adjacent layers, A is block tridiagonal in this ordering. Let v satisfy $Av = f$ with $f = 0$ on $\Gamma_2 \cup \Omega_2$. Then

$$A\,U = (0,\ A_{\Gamma_1\Gamma_2} v_{\Gamma_2},\ -A_{\Gamma_2\Gamma_1} v_{\Gamma_1},\ 0) \quad \Longrightarrow \quad U = (0,\, 0,\, v_{\Gamma_2},\, v_{\Omega_2}), \tag{2}$$

which one checks row block by row block: the Ω_1 and Γ_1 rows hold because U vanishes on $\Omega_1 \cup \Gamma_1$, the Γ_2 row is the equation $(Av)_{\Gamma_2} = 0$ rearranged, and the Ω_2 row is $(Av)_{\Omega_2} = 0$. Nothing in this uses symmetry, a Green's identity, or the fact that A discretises a differential operator. The sparsified H has a compact 3×3 stencil, so one layer on each side suffices, exactly as for the five-point Laplacian of the paper. On the assembled H, with exact traces, the identity holds to 10^{-13} for straight interfaces and for the L-shaped interface of a quarter-space.

The precedent is worth naming: Zepeda-Núñez and Zhao [3] already combined Ying's sparsifying preconditioner with the layered method of polarized traces for the 2D LS equation. The obstacle they met is instructive. Since the sparsified matrix does not come from a differential operator, no absorbing boundary condition was available for its local problems; they built one from a smooth cut-off of m with a complex shift, and found that one-directional sweeps then needed $O(\sqrt{\omega})$ iterations, which alternating directions brought back to $O(\log \omega)$. The 2018 stencils γ, fitted at the local frequency, are precisely the absorbing rows that were missing.

3.2 Local box problems from the stencils alone

A box Ω_{ij} is extended by two unstretched layers and then by p layers of PML on every side that has a neighbour; boxes on the outer boundary keep the global PML there. A row of the local matrix is decided by the stretching increments $\Delta\sigma_d$ of the point on each axis: zero increments give the α/β row of H, and any non-zero increment gives a γ row fitted at $\kappa = \omega\sqrt{1-m}$ with those increments, corner points included. The per-axis profile is the two-sided quadratic ramp already used for the global PML, with $\pm\infty$ edges on sides without a pad. Consequently a local matrix needs only α, β, m and the geometry, never H: every rank can build and factorise exactly the boxes it owns, which is the embarrassingly parallel setup the paper relies on.

Two conventions turned out to matter, each by three orders of magnitude, and both were found by getting them wrong first.

- **Where the stretching starts.** The zero of the PML profile must sit on the *first* pad row, so that the stretching begins one row inside the pad; this is what the moving PML of the layered sweep does. A closed 64^2 box with a source inside then reproduces the free-space solution to $4.4 \cdot 10^{-5}$ in a constant medium and $6.6 \cdot 10^{-4}$ inside the lens, independently of the pad width and of the source position. Placing the zero on the last interior row instead gives 14–15%.

- **The corner of an L-shaped trace.** The corner degree of freedom belongs to the diagonal neighbour and must take its value from that box. Taking it from a cardinal neighbour's polarized field, which vanishes there by construction, leaves a pad-independent error near 1% in the diagonal box; with the corner from its owner the diagonal step is as accurate as the source box.

3.3 Sources everywhere without windows

In the preconditioner the right-hand side is $f = Sr$ for a Krylov residual r, so it is non-zero at every grid point. The paper's fourth scenario, with four shifted decompositions and a partition of unity, exists because a continuous source can straddle an interface. On a grid whose interfaces run *between* grid lines no degree of freedom lies on the skeleton, so the third scenario, arbitrary sources not intersecting the skeleton, already covers every right-hand side and the factor four is not paid.

What must be paid instead is bookkeeping against double counting. The field of a box is kept as nine components: its own local solution w^0, what arrived from each cardinal direction $w^{\rightarrow}, w^{\leftarrow}, w^{\uparrow}, w^{\downarrow}$, and what arrived along each diagonal $w^{\nearrow}, w^{\nwarrow}, w^{\searrow}, w^{\swarrow}$. The trace handed on sums exactly the components that carry the sources of the relevant half-plane or quadrant. For the upward step from (i, j) into $(i{+}1, j)$ the trace is $w^0 + w^{\uparrow}$ of (i, j). For the north-east diagonal step into (i, j), the horizontal arm carries $w^{\rightarrow} + w^{\nearrow}$ of the box below, the vertical arm $w^{\uparrow} + w^{\nearrow}$ of the box to the left, and the corner $w^0 + w^{\rightarrow} + w^{\uparrow} + w^{\nearrow}$ of the diagonal box. The final field is the sum of all nine components on the owned points.

3.4 What does not transfer

The one-iteration property moves inside. In the paper the preconditioner inverts the matrix being solved, so it becomes exact as the PML thickens. Here it inverts a surrogate, and the outer iteration runs on the dense operator: the exact H^{-1} leaves 3 iterations, and thickening the absorber of the layered sweep from one to four wavelengths brings the count from 6 down to 4, not to 1. The property is available where the two-level scheme of [3] needs it: a few GMRES steps on H inside each application, preconditioned by the sweep, bring the outer count to the floor of 3. Because such a preconditioner is not a fixed linear operator, the outer solver must be a flexible GMRES that stores the preconditioned basis; with plain GMRES the iteration reports convergence while the true residual stalls near 10^{-6}.

The outer operator is global. The paper's Krylov products are sparse and local. Ours are dense: a global FFT with an all-to-all transpose, or a fast multipole product with its own tree layout. In the present version every rank keeps the full vectors and runs the outer iteration replicated, so the sweep is distributed but the dense product is not; distributing it is the next step, and the row-based box ownership is already a decomposition into horizontal bands that a slab FFT or a band-partitioned fast method can share. With a fast multipole product the FFT disappears from the solver altogether, since the preconditioner itself uses none.

4 Implementation: one dataflow for threads, MPI and both

The sweep is expressed as a dependency graph rather than as nested loops. A *node* is a pair (box, direction): the arrival of one component in one box, with direction 0 for the box's own solve. A node has *contributions*, each a set of trace points whose values are the sum of named components of one neighbouring box, available once the nodes producing those components have fired. All of this is geometric and is computed on every rank at setup. At solve time a node fires when its contributions are present; firing it runs the polarized solve (2) and releases the contributions that depend on it. If source and target boxes are on the same rank the values are copied; otherwise they travel as one message, tagged by target node and contribution, into a receive posted at the start of the solve. Within a rank a pool of tasks executes ready nodes, so rows, columns and anti-diagonals overlap as far as the dependencies allow; the calling task polls the outstanding receives. Boxes are assigned to ranks row by row, the paper's assignment, so horizontal sweeps never communicate and columns and diagonals pipeline across ranks; an arbitrary ownership matrix is accepted. The owned parts of the result are combined by one all-reduce.

Every node's arithmetic depends only on its inputs, summed in a fixed order, so sequential, threaded and distributed runs agree bitwise. The tests check this for one against eight worker tasks in one process, and for three ranks of two threads against one process. The code is in `src/lsweep.jl`; the MPI backend is the weak-dependency extension `ext/SparsifyAndSweepMPIExt.jl`, which serialises MPI calls behind a lock so that any thread level from *serialized* up suffices. It is selected with `SweepPreconditioner(P;mode=:lsweep,nbox,pad,comm,inner)`.

5 Numerical evidence

One transmission. A minimal 2×2 decomposition with 64-point boxes, the corner at the centre of the converging lens ($m \approx -1$ there), source in the lower-left box; relative error of each box against the global solution of H:

medium, $\omega/2\pi$	pad	source box	straight steps	diagonal step
constant, 32	1λ	$8 \cdot 10^{-5}$	$7.5 \cdot 10^{-4}$	$1.3 \cdot 10^{-4}$
constant, 32	2λ	$8 \cdot 10^{-5}$	$3.0 \cdot 10^{-4}$	$1.3 \cdot 10^{-4}$
lens, 32	1λ	$4 \cdot 10^{-4}$	$3.8 \cdot 10^{-3}$	$4.5 \cdot 10^{-4}$
lens, 32	2λ	$2.6 \cdot 10^{-4}$	$1.5 \cdot 10^{-3}$	$1.7 \cdot 10^{-4}$
lens, 64	1λ	$1.1 \cdot 10^{-3}$	$1.8 \cdot 10^{-2}$	$2.0 \cdot 10^{-3}$
lens, 64	2λ	$4.7 \cdot 10^{-4}$	$8.5 \cdot 10^{-3}$	$4.6 \cdot 10^{-4}$
lens, 64	3λ	$4.2 \cdot 10^{-4}$	$4.1 \cdot 10^{-3}$	$5.2 \cdot 10^{-4}$
lens, 128	1λ	$1.0 \cdot 10^{-3}$	$1.1 \cdot 10^{-2}$	$1.5 \cdot 10^{-3}$
lens, 128	2λ	$7.8 \cdot 10^{-4}$	$5.2 \cdot 10^{-3}$	$1.1 \cdot 10^{-3}$
lens, 128	3λ	$7.8 \cdot 10^{-4}$	$3.5 \cdot 10^{-3}$	$1.1 \cdot 10^{-3}$

The straight steps halve with each doubling of the absorber, the signature of a working PML; the floor near 10^{-4} is the accuracy of the fitted stencils themselves; the diagonal step is as accurate as the source box.

One full application against $H^{-1}f$ for a random f supported on all of Ω ($n = 127$, $\omega/2\pi = 16$): in a constant medium between $1.8 \cdot 10^{-4}$ and $6.3 \cdot 10^{-4}$ for 2×2 to 4×4 boxes, halving from one to two wavelengths of pad; in the lens between $2 \cdot 10^{-3}$ and $3 \cdot 10^{-2}$ depending on where the interfaces cut it, and nearly independent of the pad: as in §4 of [2], the local problems do not see reflections between boxes, and the outer iteration supplies them.

As the preconditioner. Converging lens, $h = \lambda/8$, global PML width $b = 8$, GMRES(20) to 10^{-6}, one process with 16 threads on a 16-core desktop. Boxes of about 64 points, each padded by 8 points — one wavelength at this h — unless the row says otherwise, and UMFPACK's own factors throughout; the compact factors come later in this section. Each row was measured in a process of its own, so the peak resident set belongs to that row alone:

$\omega/2\pi$	inner solver	setup	apply	iterations	solve	peak
32	layered sweep	1.08 s	0.036 s	5	0.25 s	0.7 GB
	L-sweeps	0.50 s	0.024 s	4	0.18 s	0.9 GB
	L-sweeps, inner 6	0.46 s	0.075 s	3	0.27 s	0.9 GB
64	layered sweep	1.99 s	0.131 s	6	1.03 s	1.4 GB
	L-sweeps	1.24 s	0.115 s	5	0.68 s	1.8 GB
	L-sweeps, inner 6	1.25 s	0.398 s	3	1.09 s	1.9 GB
128	layered sweep	6.22 s	0.551 s	7	4.55 s	4.1 GB
	L-sweeps	3.33 s	0.393 s	7	3.61 s	5.6 GB
	L-sweeps, pad 16	5.03 s	0.595 s	6	4.26 s	7.2 GB
	L-sweeps, inner 6	3.30 s	1.531 s	3	4.95 s	5.7 GB
256	layered sweep	29.36 s	2.059 s	9	23.99 s	15.1 GB
	L-sweeps	11.10 s	1.554 s	14	28.34 s	20.5 GB
	L-sweeps, pad 16	15.36 s	2.239 s	10	27.95 s	27.0 GB
	L-sweeps, pad 24	22.53 s	3.042 s	8	29.70 s	34.9 GB
	L-sweeps, inner 6	11.14 s	9.245 s	3	27.98 s	20.5 GB

Despite nine solves per box against two per slice, L-sweeps sets up between 2.0 and 2.6 times faster at every size — at $\omega/2\pi = 256$ it factorises 1024 local problems of 7140 unknowns where the layered sweep factorises 256 slabs of up to 51 625 — and on time to solution with setup included it wins throughout, though the margin narrows steadily: 1.95× at $\omega/2\pi = 32$, then 1.57, 1.55 and 1.35× at 256. The mechanism behind both facts is not the one the table suggests, so it is worth measuring rather than assuming.

What threads, and what does not. `src/sweep.jl` contains no threading constructs at all. The layered sweep is serial in both of its stages: its application is a sequential recurrence, which is inherent to the method, but its factorisation loop over slices writes only into per-slice arrays, so it is embarrassingly parallel and simply is not threaded. Speedups from 1 to 16 threads, with BLAS pinned to the Julia thread count so that a one-thread run cannot quietly use all sixteen cores inside UMFPACK's dense kernels:

	layered sweep		L-sweeps	
$\omega/2\pi$	factorisation	application	factorisation	application
32	1.03×	0.97×	1.10×	4.17×
64	1.03×	1.05×	2.00×	6.40×
128	0.96×	1.01×	2.30×	6.38×
256	0.93×	0.98×	3.06×	6.39×

The layered columns are flat, and slightly below one at 256. The dense operator is serial too, for a duller reason: the package never calls `FFTW.set_num_threads`, so the FFT convolution applied once per GMRES iteration costs the same at every thread count.

The layered sweep is fast regardless because it does far less work. On one core at $\omega/2\pi = 256$ one layered application costs 1.97 s against 9.01 s for one L-sweeps application, a factor of 4.6; taking each at its own iteration count, the layered sweep reaches the answer in $22.6 + 9 \cdot 1.97 = 40.3$ s against $17.9 + 14 \cdot 9.01 = 144.1$ s, which is 3.6 times faster. Serial factorisation costs are close, 22.6 s

against 17.9 s. So every L-sweeps advantage in the preceding table is bought with the other fifteen cores: the comparison is between a parallel implementation and a serial one, and what L-sweeps really trades is about 4.6× more application work for parallelism the layered implementation does not yet exploit.

The absorber has to grow with the frequency. What narrows the margin is the iteration count, at a pad held fixed in wavelengths:

$\omega/2\pi$	layered sweep	pad 1λ	pad 2λ	pad 3λ
32	5	4	4	4
64	6	5	5	5
128	7	7	6	6
256	9	14	10	8

Some growth is expected: the paper reports $N_{\mathrm{it}} = O(\log \omega)$, which over these four frequencies would read roughly $4, 5, 6, 7$. The 3λ column, $4, 5, 6, 8$, about recovers that; the 1λ column outruns it. The natural explanation — that the reflection each absorber leaves accumulates over the $q \propto \omega$ interfaces a wave crosses — is wrong, and was tested: at $\omega/2\pi = 256$ and a 1λ pad, boxes of 64, 96 and 128 points ($q = 32$, 22, 16) need 14, 14 and 15 iterations, and at 2λ the 64- and 128-point boxes both need 10. Halving the interfaces changes nothing. And 128-point boxes at 256 against 64-point boxes at 128 — the same $q = 16$, the same pad — need 15 against 7. What sets the count is the pad in wavelengths and the frequency itself, the size of Ω in wavelengths; why propagation distance should matter at fixed per-interface absorption is open. Widening the absorber to 3λ at $\omega/2\pi = 256$ restores the count to 8, below the layered sweep, at the price of a local problem of 13 572 unknowns instead of 7140: setup doubles, the application doubles, and the peak grows to 34.9 GB. For one right-hand side the narrow pad remains the better trade. For many right-hand sides, where the setup amortises and the solve phase alone decides, the layered sweep overtakes every L-sweeps variant with UMFPACK factors at 256; the compact factors of the next paragraph reverse that.

Half the bytes. If the application is bound by the bytes it streams, the lever is the factors themselves. Each box's UMFPACK factorisation is now re-stored, once at setup, as L and U in single precision with 32-bit indices — 12 bytes per nonzero against UMFPACK's 24 — together with the row scaling and permutations that reproduce its solve, $(R_s A)[p, q] = LU$. The triangular solves run in Julia, accumulate in double precision, allocate nothing, and skip the zeros a trace right-hand side leaves in the running vector. The per-box solve agrees with UMFPACK's to 10^{-6}, the preconditioner is unchanged to the digits shown, and the result still agrees bitwise across worker counts. At $\omega/2\pi = 256$:

64-point boxes unless stated	setup	apply	iterations	solve	peak
pad 8, UMFPACK factors	10.27 s	1.563 s	14	28.61 s	20.4 GB
pad 8, compact	12.13 s	1.089 s	14	19.44 s	14.5 GB
pad 16, compact	16.82 s	1.472 s	10	18.36 s	18.2 GB
pad 16, 128-point boxes, compact	17.93 s	1.069 s	10	14.85 s	15.8 GB
layered sweep	29.36 s	2.059 s	9	23.99 s	15.1 GB

The application is 1.44× faster and the peak 29% lower, for two seconds of extraction at setup; the gain is short of the 2× in bytes because the solves are latency-bound per core as well as bandwidth-bound in aggregate. The solve phase is again ahead of the layered sweep's, 19.4 s against

24.0 s, and time to solution is 31.6 s against 53.4. Larger boxes bought no iterations but do buy bytes — the padded box's redundancy falls from 1.72× at 64 points to 1.34× at 128 — which is why 128-point boxes with a 2λ pad give the fastest solve phase measured, 14.9 s. Two smaller changes to the setup went in alongside. BLAS is pinned to one thread while the boxes factorise, since its threads only spun at their barriers (92 CPU-seconds of 150); the pin is unconditional, because the BLAS thread count changes the factors in their last bits and one-worker and sixteen-worker runs are meant to agree bitwise, and it is the same fix as the `OPENBLAS_NUM_THREADS=1` of the next table, applied inside. And the local matrix is assembled straight into CSC rather than through a coordinate list and `sparse()`, bitwise the same matrix for two thirds of the allocation. Together they take the setup from 11.1 to 10.3 s here.

Memory. A box padded by one wavelength carries 7140 unknowns against the 4160 it owns, a 1.7× redundancy. With UMFPACK's factors the measured peak is about 1.35× the layered sweep's at every size; with the compact ones it is 14.5 GB against 15.1 at $\omega/2\pi = 256$, below it. At $\omega/2\pi = 512$ ($N = 1.7 \cdot 10^7$) neither method fits the 61 GB of this machine: both were stopped by a watchdog at 40 GB while still in setup, and scaling the 256 peaks by the fourfold growth in N puts them near 65 GB for the layered sweep and 58 for L-sweeps with compact factors — closer, but not this machine. Distributing the boxes is what lifts that ceiling, which is the subject of the next table.

Threads, ranks and both. Setup and one application at $\omega/2\pi = 256$ (1024 boxes), best of three, ranks bound to disjoint cores, timed as the slowest rank:

configuration	setup	apply	peak per rank
1 process × 16 threads	6.80 s	1.43 s	15.1 GB
4 ranks × 4 threads	4.54 s	1.62 s	4.7 GB
8 ranks × 2 threads	4.77 s	1.98 s	2.9 GB
16 ranks × 1 thread	4.48 s	1.69 s	2.1 GB

The messages are two grid layers per interface and hide behind the local solves: the application is flat across every configuration. Setup is flat too across the distributed ones and half again slower with threads alone, which is the sparse LU's poor threading showing through once more. The column that matters is the last: memory per rank falls sevenfold from one process to sixteen, because the boxes are partitioned and only the grid-sized vectors and the kernel are replicated. That is what puts problems larger than one node can hold — $\omega/2\pi = 512$ among them — within reach of a cluster.

Two configuration traps are worth recording, both of which cost a factor of three. Without core binding the 4×4 run applies in 1.18 s, because the launcher pins each rank to a single core and its threads then share it. And with OpenBLAS left at its default thread count, the 4×4 setup takes 13.5 s instead of 4.5 s: every rank starts a full thread pool inside its four cores and the pools oversubscribe. The numbers above are with `OPENBLAS_NUM_THREADS=1`, which is the right setting whenever more than one rank shares a node.

A constant-contrast disc, against the Mie series. A disc of radius 1/4 at $(1/2, 1/2)$ — the unit disc after rescaling lengths by four, so $\omega R = \omega/4$ — with velocity 0.8 inside (index 1.25) and 1 outside, under the same plane wave and otherwise the setup of the table above. The package solves for the scattered field, and the Mie series gives it exactly; every row at a frequency solves

the same discrete system to 10^{-6}, so the Mie error is the discretisation's and is the same on every row. Below $\omega/2\pi = 16$ the 64-point box is the whole grid and the L-sweeps rows are the exact solve of H in single precision; "exact solve of H" is its UMFPACK LU.

$\omega/2\pi$	inner solver	setup	apply	outer	sweeps	fwd	Hmv	t_{sw}	t_{fwd}	t_{Hmv}	solve	peak
2	layered sweep	0.33 s	0.000 s	4	5	4	0	0.0000 s	0.0000 s	0.0000 s	0.02 s	0.5 GB
	one box (= exact H)	0.09 s	0.000 s	2	3	2	0	0.0000 s	0.0000 s	0.0000 s	0.02 s	0.6 GB
	one box, inner 6	0.10 s	0.001 s	2	4	3	2	0.0005 s	0.0000 s	0.0000 s	0.00 s	0.6 GB
4	layered sweep	0.49 s	0.001 s	6	7	6	0	0.0010 s	0.0000 s	0.0000 s	0.02 s	0.5 GB
	one box (= exact H)	0.11 s	0.000 s	3	4	3	0	0.0000 s	0.0000 s	0.0000 s	0.02 s	0.6 GB
	one box, inner 6	0.10 s	0.001 s	3	6	4	3	0.0005 s	0.0000 s	0.0000 s	0.00 s	0.6 GB
8	layered sweep	0.63 s	0.003 s	9	10	9	0	0.0030 s	0.0001 s	0.0000 s	0.04 s	0.6 GB
	one box (= exact H)	0.13 s	0.001 s	3	4	3	0	0.0010 s	0.0001 s	0.0000 s	0.02 s	0.6 GB
	one box, inner 6	0.14 s	0.003 s	3	6	4	3	0.0015 s	0.0001 s	0.0000 s	0.02 s	0.6 GB
16	layered sweep	0.75 s	0.009 s	12	13	12	0	0.0090 s	0.0005 s	0.0001 s	0.14 s	0.6 GB
	L-sweeps pad 8	0.22 s	0.003 s	12	13	12	0	0.0030 s	0.0005 s	0.0001 s	0.09 s	0.7 GB
	L-sweeps pad 8, inner 6	0.22 s	0.058 s	3	24	4	21	0.0073 s	0.0005 s	0.0001 s	0.08 s	0.7 GB
32	exact solve of H	0.47 s	0.018 s	3	4	3	0	0.018 s	0.0021 s	0.0005 s	0.11 s	0.7 GB
	layered sweep	1.05 s	0.039 s	16	17	16	0	0.039 s	0.0021 s	0.0005 s	0.62 s	0.8 GB
	L-sweeps pad 8	0.47 s	0.011 s	19	20	19	0	0.011 s	0.0021 s	0.0005 s	0.40 s	1.2 GB
	L-sweeps pad 8, inner 6	0.48 s	0.127 s	5	40	6	35	0.016 s	0.0021 s	0.0005 s	0.54 s	1.1 GB
64	exact solve of H	1.60 s	0.069 s	4	5	4	0	0.069 s	0.0092 s	0.0029 s	0.47 s	1.4 GB
	layered sweep	1.98 s	0.136 s	34	36	35	0	0.136 s	0.0092 s	0.0029 s	5.38 s	1.5 GB
	L-sweeps pad 8	1.23 s	0.066 s	32	34	33	0	0.066 s	0.0092 s	0.0029 s	2.72 s	1.7 GB
	L-sweeps pad 8, inner 6	1.20 s	0.481 s	6	48	7	42	0.060 s	0.0092 s	0.0029 s	3.13 s	1.9 GB
128	exact solve of H	7.06 s	0.304 s	5	6	5	0	0.304 s	0.055 s	0.012 s	2.19 s	4.4 GB
	layered sweep	6.30 s	0.523 s	51	54	53	0	0.523 s	0.055 s	0.012 s	31.50 s	4.5 GB
	L-sweeps pad 8	3.62 s	0.312 s	53	56	55	0	0.312 s	0.055 s	0.012 s	17.97 s	4.3 GB
	L-sweeps pad 16	5.22 s	0.366 s	53	56	55	0	0.366 s	0.055 s	0.012 s	22.77 s	5.4 GB
	L-sweeps pad 8, inner 6	3.65 s	1.975 s	8	64	9	56	0.247 s	0.055 s	0.012 s	16.21 s	4.6 GB
256	exact solve of H	31.91 s	1.264 s	10	11	10	0	1.264 s	0.308 s	0.046 s	18.25 s	16.6 GB
	layered sweep	31.40 s	2.082 s	156	164	163	0	2.082 s	0.308 s	0.046 s	403.49 s	16.4 GB
	L-sweeps pad 8	12.46 s	1.100 s	328	345	344	0	1.100 s	0.308 s	0.046 s	457.84 s	14.6 GB
	L-sweeps pad 16	16.97 s	1.552 s	317	333	332	0	1.552 s	0.308 s	0.046 s	566.54 s	18.4 GB
	L-sweeps pad 24	23.72 s	1.997 s	297	312	311	0	1.997 s	0.308 s	0.046 s	660.11 s	23.3 GB
	L-sweeps pad 8, inner 6	12.40 s	7.673 s	22	176	24	154	0.959 s	0.308 s	0.046 s	177.83 s	18.3 GB

Columns. $\omega/2\pi$: frequency; the grid is $n = 8(\omega/2\pi) - 1$ at eight points per wavelength and the disc is $\omega/4\pi$ wavelengths across. *Inner solver*: the approximate inverse of H the preconditioner uses; "inner 6" runs up to six GMRES iterations on H per application, to 10^{-3}, in place of a single sweep. *Setup*: building it — sparsification, H, and the factorisation. *Apply*: one preconditioner application, timed once, comprising Sr, the inner solve and the scatter. *Outer*: iterations of the outer Krylov solver on the dense system. *Sweeps*: preconditioner applications performed inside that solve, LU solves on the exact-H row. *Fwd*: forward maps, applications of $A = I + \omega^2 K[m\,\cdot\,]$, each an FFT convolution. *Hmv*: applications — not solves — of the sparsified H, one per inner GMRES iteration; only the two-level rows perform any. t_{sw}: the cost of one sweep, namely *apply* divided by the sweeps that application performs (one for the single-level rows, eight for the two-level rows from $\omega/2\pi = 16$ up); it carries Sr and the scatter with it, 0.050 s of 1.100 at $n = 2047$. t_{fwd}, t_{Hmv}: the cost of one forward map and of one H matvec. Those two depend on the grid alone — not on the medium, not on the preconditioner — so one measurement per size serves every row; t_{Hmv} is already inside the two-level rows' t_{sw}. *Solve*: the outer Krylov solve, setup excluded. *Peak*: peak resident set of the process. Every column is measured.

The Mie error of the scattered field, relative ℓ^2 on 20 000 grid points at eight points per wavelength, is 0.16, 0.10, 0.074, 0.076, 0.083, 0.20, 0.45 and 0.56 for $\omega/2\pi = 2$ to 256; at fixed $\omega/2\pi = 8$ it falls as 0.074, 0.026, 0.0090, 0.0045 at 8, 16, 32, 64 points per wavelength — first order in h, as a staircased discontinuous coefficient gives. So the solvers agree with each other to their tolerance and the error is the discretisation's: at a fixed resolution it grows with ω because the staircased interface shifts the effective radius by about $h/2$ and a large, near-resonant scatterer is sensitive to that. The corrected trapezoidal rule is built for smooth m; a sharp interface costs it about 64 points per wavelength for 5% at $\omega/2\pi = 256$.

The counts follow the solvers' schedule. A left-preconditioned GMRES(20) applies operator and

preconditioner once per iteration and once more at every restart, plus one preconditioner application on the initial residual, so the single-level rows have fwd = outer + r and sweeps = outer + 1 + r with $r = \lfloor(\text{outer} - 1)/20\rfloor$ restarts, and the two-level rows have Hmv = sweeps − outer. Only the two-level sweep counts cannot be derived: each application runs an inner GMRES that stops at 10^{-3} or its cap of six, which is two sweeps per application at $\omega/2\pi \leq 8$ and eight from 16 upward. As a check, sweeps $\times\, t_{\text{sw}} + \text{fwd} \times t_{\text{fwd}}$ accounts for 392 s of the layered sweep's 403 at $\omega/2\pi = 256$, 176 of the two-level scheme's 178 and 17.0 of the exact solve's 18.3; it overshoots the single-level L-sweeps rows by some 8%, so the single timed application those rows report is a little pessimistic against the same sweep run back to back inside a solve. At $n = 2047$ a forward map costs 6.6 times an H matvec, 0.308 s against 0.046, and that ratio is the whole reason the two-level scheme pays.

The iterations tell a different story from the lens. The exact solve of H needs 3, 4, 5, 10 iterations at $\omega/2\pi = 32$ to 256: the sparsification's floor is modest. Everything above it is the sweeps' inexactness on a medium that reflects. The lens reflects weakly and one pass, layered or L-shaped, captures almost the whole field; the disc's interface sends waves back, each pass captures one more reflection, and the layered sweep needs 156 and the L-sweep 328 — with GMRES(20), eight and sixteen restarts. It is not the absorber: a 3λ pad takes 328 to 297 while doubling the application cost, where on the lens it took 14 to 8. Restarts cost too: 328 iterations of GMRES(20) carry sixteen restarts, each an extra forward map and sweep. The work columns show why the two-level scheme, useless on the lens, is decisive here. It does not sweep less — 176 against the layered sweep's 164 — but its sweeps are L-sweeps at about 0.96 s rather than layered sweeps at 2.08 s, and it cuts the forward maps from 163 to 24; against single-level L-sweeps, 176 sweeps against 345 and 24 forward maps against 344. The inner iterations move the work onto the sparse system H, where an iteration costs a sweep and a sparse matvec, and off the dense system, where one costs a forward map plus orthogonalisation against a basis of 4.2-million-entry vectors: 190 s to solution against 435 and 470. The exact LU of H is the limit of that trade and beats all of them — ten forward maps, eleven LU solves, 31.9 s of setup against the layered sweep's 31.4, 50 s in total at the same 16.6 GB. In two dimensions at these sizes the factorisation the sweeps exist to avoid is not expensive, and on a medium that reflects, avoiding it costs an order of magnitude. The sweeps earn their keep where that factorisation is prohibitive, which is three dimensions, and where the medium lets one pass suffice.

Three dimensions. The paper's own 3D construction is now in: a $q \times q$ decomposition of the (x_1, x_2) plane, every box spanning the whole of x_3 with the global PML at its ends — their "beam-shaped quasi-one-dimensional local problems" — with the same eight directions and the same dataflow; trace layers become planes and the diagonal trace an L-shaped prism. The scale differs: their Table 9 runs $q = 12$ to 32 beams per side on $p = q$ nodes, in constant media at six points per wavelength with 2λ boxes and a 1λ PML; here 3×3 and 4×4 beams on one node, 2.25λ boxes and a 0.5λ pad, in the lens. Converging lens, $h = \lambda/8$, $b = 4$, $C = 12$ as in the package's 3D tables; L-sweeps on 16 threads with compact factors, the layered sweep untouched — its application on one core, its 3D factorisation using OpenBLAS's default 16 threads in UMFPACK's dense fronts, which are large enough here to matter (7.5 s of setup with one BLAS thread against 5.0 s with sixteen at $n = 31$, unlike 2D where they only spun). "In.×out." is the nominal count outer × max(inner, 1); "sweeps" is the number of sweep applications actually performed inside the outer solve, and differs from it in both directions. A left-preconditioned GMRES applies its preconditioner to the initial residual as well, so the inner-free rows sweep outer+1 times; with inner iterations each of the outer applications is an inner GMRES that stops at 10^{-3} after two iterations — three sweeps — so those rows sweep about 3×outer times, half of inner×outer:

$\omega/2\pi$	inner solver	setup	apply	outer	in.×out.	sweeps	solve	peak
4	layered sweep	4.93 s	0.223 s	4	4	5	1.20 s	2.5 GB
	layered sweep, inner 6	4.97 s	0.716 s	3	18	9	2.21 s	2.4 GB
	L-sweeps 3×3, pad 4	10.54 s	0.161 s	3	3	4	0.74 s	8.2 GB
	L-sweeps 3×3, pad 4, inner 6	12.11 s	0.345 s	3	18	8	1.33 s	8.1 GB
8	layered sweep	28.26 s	1.748 s	4	4	5	8.89 s	13.3 GB
	layered sweep, inner 6	28.87 s	5.218 s	3	18	9	15.97 s	13.4 GB
	L-sweeps 4×4, pad 4	169.90 s	1.628 s	4	4	5	8.36 s	39.9 GB
	L-sweeps 4×4, pad 4, inner 6	162.59 s	4.926 s	3	18	9	15.01 s	40.6 GB

Inner iterations do not pay: the outer count sits at or next to the exact-H floor of three without them, so they only multiply the application cost. At $n = 31$ L-sweeps is the better inner solver — three iterations against four, the application 1.4× and the solve phase 1.6× faster — at twice the setup and three times the memory. At $n = 63$ the layered sweep wins: the same four iterations, an application 7% faster, the solve phase within 6%, and a setup six times slower at three times the memory. The reasons are geometric. A padded beam carries $(s + 2(2 + \text{pad}))^2\, n_{\text{tot}}$ unknowns for $s^2 n_{\text{tot}}$ owned — 3.1× redundancy at $s = 16$, pad $= 4$, against 1.7× for the 2D default — 3D sparse LU cost grows like N^2 rather than $N^{3/2}$, and pad 4 is already half a wavelength, so there is no thinner absorber to reach for. The paper's estimate says the same — 3D factorisation scales like $n^3 n_{\text{pml}}^6/p$, "therefore it is crucial to keep the thickness of the PML region as thin as possible" — and its own 3D applications take 52 s at $N = 156^3$ on twelve nodes. A factorisation in flight holds about four times its final compact size in UMFPACK transients; sixteen beams at once passed 45 GB and were stopped, and capping them at eight (`nfact=8`) holds the peak to 39.9 GB in two rounds.

6 Outlook

The paper demonstrates $O(N/p)$ scaling with one rank per row of subdomains, up to 128 ranks at 12 928 points per side. On this one node the comparison in §5 is not yet a fair fight: L-sweeps wins on total time at every size measured, 1.96× at $\omega/2\pi = 32$ falling to 1.35× at 256, but it wins by using sixteen cores against a layered implementation that uses one. With the compact factors the margin at 256 is 1.69× and the solve phase is ahead as well. In three dimensions the beam variant loses to the layered sweep at $n = 63$ on setup and memory, for the geometric reasons given in §5. And on a medium that reflects — the constant-contrast disc — both sweeps need well over a hundred iterations at 256 where the exact LU of H needs ten and costs no more to set up than the layered sweep: in two dimensions the case for sweeping at all rests on the medium being smooth. Four things remain. The outer product should be distributed in the same horizontal bands as the boxes, by a slab FFT or by a fast multipole method, after which the all-reduce of the result disappears and only the Krylov inner products are global. The row partition caps the useful rank count at the number of box rows; a two-dimensional block partition is accepted by the code but its communication pattern has not been tuned. The layered sweep's factorisation should be threaded before any of this is measured again: its slice loop is independent per slice, and at the 3.1× L-sweeps manages here it would fall from 24.2 s to about 7.3 s at $\omega/2\pi = 256$, enough to make the layered sweep the faster method outright, since its solve phase already is. Until that is done, §5 compares implementations rather than algorithms. And the absorber wants a rule: held fixed in wavelengths it lets the iteration count grow about linearly with the frequency at one wavelength and roughly logarithmically at three. The box-size study rules out the interface count as the driver, which leaves the size of the domain in wavelengths, and no mechanism for that yet.

A rule for the pad, and the reason behind it, is the first thing worth measuring on a machine large enough to reach $\omega/2\pi = 512$ and beyond.

References

[1] F. Liu, L. Ying, Sparsify and sweep: an efficient preconditioner for the Lippmann–Schwinger equation, SIAM J. Sci. Comput. 40 (2018) B379–B404.

[2] M. Taus, L. Zepeda-Núñez, R. J. Hewett, L. Demanet, L-Sweeps: a scalable, parallel preconditioner for the high-frequency Helmholtz equation, J. Comput. Phys. 420 (2020) 109706.

[3] L. Zepeda-Núñez, H. Zhao, Fast alternating bidirectional preconditioner for the 2D high-frequency Lippmann–Schwinger equation, SIAM J. Sci. Comput. 38 (2016) B866–B888.

[4] L. Ying, Sparsifying preconditioner for the Lippmann–Schwinger equation, Multiscale Model. Simul. 13 (2015) 644–660.